

Tutorial Backend - Módulo Grupos

Sistema de votaciones - 7.º Programación

Objetivo del grupo

Implementar todo lo necesario para crear, consultar, actualizar y eliminar los grupos participantes y la información básica que el frontend mostrará de cada proyecto.

Estructura general del backend

```
backend/  
├── src/  
│   ├── config/  
│   │   └── db.js  
│   ├── controllers/  
│   ├── models/  
│   ├── routes/  
│   ├── services/  
│   └── app.js  
├── .env  
├── .env.example  
├── .gitignore  
├── package.json  
└── README.md
```

Cada grupo debe modificar únicamente los archivos correspondientes a su recurso, salvo que sea necesario coordinar un cambio común con el resto del equipo.

Flujo obligatorio

Frontend → Route → Controller → Service → Model → MongoDB

- Route: define la URL y llama al controller.
- Controller: lee req.params, req.query o req.body; llama al service y responde con res.
- Service: contiene validaciones y lógica del recurso.
- Model: define el esquema de Mongoose y realiza las operaciones sobre MongoDB.
- El controller no consulta modelos directamente.
- El service no utiliza req ni res.

Archivos asignados

- src/models/Grupo.js
- src/services/gruposService.js
- src/controllers/gruposController.js
- src/routes/gruposRoutes.js

Convenciones que deben respetarse

- Todo el código y los nombres propios del proyecto estarán en español.
- Variables y funciones: camelCase.
- Modelos: singular y PascalCase, por ejemplo Grupo.js.
- Controllers: plural + Controller, por ejemplo gruposController.js.
- Routes: plural + Routes, por ejemplo gruposRoutes.js.
- Services: plural + Service, por ejemplo gruposService.js.
- URLs: minúsculas y sustantivos en plural.
- Usar const por defecto, let solo cuando el valor cambie.
- Comillas simples, punto y coma e indentación de dos espacios.
- Usar async/await para operaciones asíncronas.
- Exportar con module.exports.
- No guardar claves ni conexiones privadas dentro del código.

Modelo de datos inicial

Los campos siguientes son una base de trabajo. Si durante el desarrollo se necesita agregar o cambiar un campo, debe acordarse antes con el equipo para no romper la integración.

- **nombre:** Nombre del grupo o proyecto.
- **integrantes:** Lista simple con los integrantes del grupo.
- **descripcion:** Descripción breve del proyecto.
- **historia:** Breve explicación de cómo surgió o se desarrolló.
- **imagen:** URL o referencia de imagen; puede quedar opcional inicialmente.

Métodos que debe implementar el grupo

Método	Responsabilidad
obtenerGrupos	Obtener la lista completa de grupos.
obtenerGrupoPorId	Obtener un grupo específico por su ID.
crearGrupo	Registrar un grupo nuevo.
actualizarGrupo	Modificar los datos de un grupo.
eliminarGrupo	Eliminar un grupo.

Rutas que deben quedar disponibles

HTTP	Ruta	Controller
GET	/api/grupos	obtenerGrupos
GET	/api/grupos/:id	obtenerGrupoPorId
POST	/api/grupos	crearGrupo
PUT	/api/grupos/:id	actualizarGrupo
DELETE	/api/grupos/:id	eliminarGrupo

Orden recomendado de trabajo

Paso	Archivo	Qué debe hacerse
1	models/Grupo.js	Definir el esquema y exportar el modelo de Mongoose.
2	services/gruposService.js	Implementar operaciones usando find, findById, create,

		findByIdAndUpdate y findByIdAndDelete.
3	controllers/gruposController.js	Leer datos de Express, llamar al service y devolver códigos HTTP claros.
4	routes/gruposRoutes.js	Crear las cinco rutas acordadas y asociarlas al controller.
5	src/app.js	Coordinar con el responsable general para registrar gruposRoutes si todavía no está conectado.

Ejemplos base

Estos fragmentos sirven como referencia de estructura. Deben adaptarse a los campos definitivos y a las validaciones que acuerde el equipo.

Modelo

```
const mongoose = require('mongoose');

const grupoSchema = new mongoose.Schema({
  nombre: { type: String, required: true, trim: true },
  integrantes: [{ type: String, trim: true }],
  descripcion: { type: String, default: " " },
  historia: { type: String, default: " " },
  imagen: { type: String, default: " " }
}, { timestamps: true });

module.exports = mongoose.model('Grupo', grupoSchema);
```

Service

```
const Grupo = require('../models/Grupo');

async function obtenerGrupos() {
  return Grupo.find();
}

async function obtenerGrupoPorId(id) {
  return Grupo.findById(id);
}

module.exports = {
  obtenerGrupos,
  obtenerGrupoPorId
};
```

Controller

```
const gruposService = require('../services/gruposService');
```

```
async function obtenerGrupos(req, res, next) {  
  try {  
    const grupos = await gruposService.obtenerGrupos();  
    return res.status(200).json(grupos);  
  } catch (error) {  
    return next(error);  
  }  
}
```

```
module.exports = { obtenerGrupos };
```

Routes

```
const express = require('express');  
const gruposController = require('../controllers/gruposController');
```

```
const router = express.Router();
```

```
router.get('/', gruposController.obtenerGrupos);  
router.get('/:id', gruposController.obtenerGrupoPorId);  
router.post('/', gruposController.crearGrupo);  
router.put('/:id', gruposController.actualizarGrupo);  
router.delete('/:id', gruposController.eliminarGrupo);
```

```
module.exports = router;
```

Pruebas mínimas antes de entregar

- GET /api/grupos devuelve una lista.
- GET /api/grupos/:id devuelve 404 si el grupo no existe.
- POST rechaza datos obligatorios faltantes.
- PUT actualiza un grupo existente.
- DELETE elimina un grupo existente.
- Los controllers no importan Grupo.js directamente.

Criterio de finalización

La parte asignada se considera lista cuando sus rutas responden correctamente, los errores básicos están controlados, los nombres coinciden con este documento y no se modificaron archivos pertenecientes a otros módulos sin coordinación.